

Syrudas AI: A Local-First AI Workspace with Pluggable Model Providers

Version 0.3.0 · July 2026 · Len · MIT License

Abstract

Syrudas AI is a self-hosted AI workspace for Windows that runs entirely on the user's own machine. It provides streaming chat, an autonomous agent mode with tool use, Model Context Protocol (MCP) support, file attachments, and editor integration — while remaining radically neutral about *which* language model does the thinking. Every model backend, local or hosted, connects through a small provider-plugin contract; the rest of the application never knows which backend is talking. This paper describes the motivation, architecture, and security model of the system, and the design decisions that keep a one-person-maintainable codebase genuinely extensible.

1. Motivation

Interacting with large language models increasingly means renting access to a vertically integrated stack: one vendor's model, behind one vendor's app, with conversation history living on one vendor's servers. Projects such as PewDiePie's **Odysseus** (2026) demonstrated broad appetite for an alternative: a workspace you run yourself, where conversations, keys, and files never leave your machine.

Syrudas AI starts from the same local-first premise but elevates one idea to an organizing principle: **the model is a plugin**. Model backends appear and disappear monthly — a workspace should outlive all of them. Concretely, the goals are:

1. **Any model.** Local weights through Ollama or LM Studio, hosted frontier models through OpenRouter or first-party APIs — one UI, one history, one agent runtime across all of them.
2. **Local-first, private by construction.** The server binds to `127.0.0.1` only. There is no telemetry, no account, and no cloud component. API keys are stored locally and masked in every API response.
3. **Agentic, but consent-gated.** The model can plan and act — run commands, read and write files, search the web — under an explicit, per-action human approval model for the dangerous parts.
4. **A hub, not a silo.** The workspace speaks the OpenAI wire dialect in both directions, so external tools (editors, scripts, other apps) can use the workspace's configured models as a service.
5. **Shippable.** A single portable executable a non-developer can unzip and double-click, with zero-configuration onboarding when a local model server is already running.

2. System overview

Syrudas AI is a two-tier application:

- **Backend:** Python 3.13 / FastAPI. Serves a REST + streaming API, owns the

SQLite database, runs the agent loop, and hosts the provider and MCP subsystems.

- **Frontend:** React 19 + TypeScript (Vite). A single-page chat interface

compiled to static assets and served by the backend — the same bundle in the browser, the desktop window, and the VS Code panel.

The packaged desktop app wraps both in a native window (pywebview over WebView2), with the server running on a background thread of the same process. State is deliberately boring: one SQLite file for conversations, messages, provider instances, MCP registrations, and settings; one workspace folder for agent file output. In the portable build both live next to the executable, so copying the folder copies the installation.

```
+----- SyrdasAI.exe -----+
| pywebview window (WebView2) |
|-- React SPA -- NDJSON/HTTP --> FastAPI (127.0.0.1:8040) |
|                               |-- /api/chat · conversations |
|                               |-- /api/attachments · settings |
|                               |-- /v1 (OpenAI-compatible hub) |
|                               |-- Agent loop -- builtin tools |
|                               | +----- MCP client --> stdio servers |
|                               |-- Provider registry |
|                               |-- openai_compat (builtin) |
|                               |-- plugins/*.py (drop-in) |
| SQLite (syrdas.db) · data/workspace · plugins/ |
+-----+
| ^                                     ^ |
| | VS Code extension (panel,         | Continue / any OpenAI-   |
| | ask-about-selection)              | compatible client via /v1 |
```

3. The provider abstraction

The core contract is one small class. A provider *type* is a Python class; a provider *instance* is that class configured with user data (URL, key) in the settings UI and stored in SQLite.

```
class ModelProvider(ABC):
    type_id: str  # "openai_compat", "anthropic", ...
    display_name: str
    config_fields: list[ConfigField]  # drives the settings form

    async def list_models(self) -> list[ModelInfo]: ...
    async def chat(self, model, messages, tools=None, params=None)
        -> AsyncIterator[StreamEvent]: ...
```

Three design decisions matter here:

A normalized interior. Messages, tool specifications, tool calls, and stream events are defined once (`server/schemas.py`) in an OpenAI-flavored shape. Adapters translate at the edge; the chat pipeline, agent loop, persistence layer, and UI are all provider-agnostic. Adding Anthropic or Gemini support means writing one translation file, not touching the app.

Streaming as the only mode. `chat()` yields a flat event stream — `text_delta`, `tool_call`, `usage`, `error`, `done`. Non-streaming responses are just a stream that ends quickly, so there is exactly one code path.

Discovery by drop-in. The registry imports builtin adapters statically (a hard requirement under PyInstaller, whose bundles are invisible to `pkgutil` directory scanning) and additionally scans a user-writable `plugins/` folder next to the executable. A user can add a new backend to the *packaged* app by dropping a `.py` file beside it — no rebuild.

The single builtin adapter, `openai_compat`, covers most of the ecosystem in practice: Ollama, LM Studio, llama.cpp server, vLLM, OpenRouter, and OpenAI itself all speak the same dialect.

4. Chat pipeline

`POST /api/chat` accepts a message, persists it, and returns a streamed NDJSON body — one JSON event per line, consumed by the frontend with a `ReadableStream` reader. NDJSON-over-POST was chosen over WebSockets (no connection lifecycle to manage) and over EventSource (which cannot POST). Interleaved event types let a single stream carry plain chat, agent tool activity, and approval requests uniformly.

Messages are persisted as they complete, so a conversation survives interrupted streams, application restarts, and version upgrades. Titles derive from the first user message; attached-file blocks are stripped first.

5. Agent mode

Agent mode turns the chat into a plan-act loop: the model receives tool schemas alongside the conversation; returned tool calls are executed and their results appended; the loop continues until the model stops calling tools or a step ceiling (15) is reached.

Builtin tools:

Tool	Capability	Guard
<code>shell</code>	PowerShell command	Per-call human approval
<code>file_read / file_write / file_list</code>	Text file I/O	Path sandbox
<code>web_fetch</code>	URL → readable text	—
<code>web_search</code>	DuckDuckGo search	—

The approval gate. The HTTP stream is one-way, so consent arrives out-of-band: when the agent proposes a shell command, the loop emits an `approval_required` event, parks on an `asyncio.Future`, and resumes only when the user clicks Approve or Deny in the UI (`POST /api/approvals/{id}`). A denial is reported to the model as an ordinary tool result, so it can adjust course rather than crash. Approvals are per-call: there is no "always allow."

The file sandbox. Relative paths resolve inside a dedicated workspace folder. Absolute paths are honored only inside folders the user has explicitly granted in Settings; everything else is refused with an error the model can read. Grants are stored as data, surfaced to the model in its system prompt, and revocable at any time.

MCP. Users can register stdio MCP servers (command + arguments). Their tools are namespaced (`filesystem_read_file`) and merged into the agent's toolset. One implementation subtlety is worth recording: each server connection is owned by a dedicated `asyncio` task, because anyio cancel scopes must be entered and exited by the same task; other tasks only use the initialized session object.

6. Interoperability

Syrudas is both a *client* of the OpenAI dialect (through `openai_compat`) and a *server* of it. The `/v1` surface exposes every model of every configured provider instance under a namespaced id (`ollama-local/llama3.1:8b`), with `chat/completions` translating each request to the normalized schema and routing it to the right backend — streaming, tool calls, and usage included. These calls are stateless and never touch conversation history.

This inverts the usual integration burden. Instead of Syrudas implementing an editor plugin for every tool, any OpenAI-compatible tool can adopt Syrudas as its backend by changing a base URL. The Continue extension for VS Code is the reference consumer.

A dedicated VS Code extension complements the hub: it embeds the full workspace UI in an editor panel and adds a right-click *Ask About Selection* command that prefills a chat with the selected code (file, line, language fence) via a `?prompt=` deep-link parameter.

7. File attachments

Attachments follow a deliberately stateless pipeline: the client uploads a file to `POST /api/attachments`, the server extracts text (UTF-8 for text-like formats; `pypdf` for PDFs; binaries rejected; 120k-character cap with explicit truncation flags), and returns it. The client embeds the text in the outgoing message inside `<file name="...">` delimiters.

Storing attachment content *in the message itself* — rather than in a side-table of blobs — buys three properties at once: history replay to any provider needs no special casing, export/backup is just the database file, and the UI can reconstruct file chips (collapsible, per-file) from the message text alone. The trade-off, larger message rows, is acceptable at the capped sizes.

8. Security and privacy model

The threat model is honest about what a local, single-user tool can promise.

- **Network exposure:** the server binds to `127.0.0.1` exclusively. Nothing listens on external interfaces; there is no authentication layer because there is no remote surface.
- **Data at rest:** conversations, settings, and provider API keys live in a local SQLite file. Keys are stored in plaintext (an OS keychain integration is future work) but are masked (`•••1234`) in every API response, so the browser/UI layer never re-receives full secrets.
- **Model-initiated actions:** the dangerous capability — arbitrary shell — is gated per call with no persistent allow. File access is deny-by-default outside the workspace plus explicit grants. Web tools are unauthenticated readers.
- **Telemetry:** none. The application makes outbound connections only to model backends the user configured and URLs the user (or their agent, under the rules above) requests.
- **Residual risks:** a granted folder is fully readable/writable by any

model the user runs, including a badly aligned local one; MCP servers run with user privileges and are trusted by registration; the unsigned executable requires a SmartScreen click-through, which trains no good habits. These are documented rather than hidden.

9. Packaging and distribution

The release artifact is a portable zip: one PyInstaller onefile executable (windowed, WebView2), an end-user README, and the MIT license. Everything mutable lives beside the executable, making installations trivially copyable, movable, and deletable. On first run with an empty database, the server probes well-known local backends (Ollama on :11434, LM Studio on :1234) and auto-configures any that respond — a fresh user with Ollama running reaches a working chat with zero configuration steps.

Version identity is stamped in three places from one source of truth (APP_VERSION): the health endpoint, the UI footer, and the Windows file properties of the executable.

Two Windows-specific build lessons are preserved in the codebase for posterity: PyInstaller bundles are invisible to `pkgutil` (hence static builtin imports), and PowerShell 5.1 under `$ErrorActionPreference = "Stop"` converts native `stderr` log lines into terminating errors (hence `cmd /c "... 2>&1"` wrappers in the build scripts).

10. Limitations and roadmap

Current limitations: single-user by design; Windows-only packaging (the server itself is portable Python); no retrieval/embedding layer, so attachments are bounded by model context; text-only multimodality; plaintext key storage.

Planned directions, in rough order of value: native Anthropic and Gemini provider plugins (exercising the plugin contract beyond the OpenAI dialect); deep-research runs (multi-step search-read-synthesize with citations); a blind model arena for side-by-side evaluation built on the multi-provider core; retrieval over attachments and granted folders using local embeddings; notes/documents with AI editing.

11. Conclusion

Syrudas AI demonstrates that a genuinely useful AI workspace — streaming chat, consent-gated agency, protocol interoperability, editor integration, one-click distribution — fits in a small, single-maintainer codebase when one abstraction is chosen carefully and enforced everywhere. Models will keep changing. A workspace whose only opinion about models is a five-method contract is positioned to outlast any of them.

Syrudas AI is open source under the MIT license. Architecture reference: README.md and docs/SETUP.md in the repository.